

Stock Radar — Build Handoff & PRD

Stock Radar — Build Handoff & PRD (for Claude Code)

Date: 2026-06-24 · **Owner:** Hume **Purpose:** Hand the next chunk of work to Claude Code with full context, precise requirements, a prioritized todo list, and QA/acceptance criteria. Read this top-to-bottom before touching code.

0. Ground rules (non-negotiable — these reflect decisions already made)

1. **One atomic, falsifiable change per iteration.** Each change has its own test and is verifiable in isolation. Never bundle (a prior 17-fix bundle doubled an error). Commit per milestone with a `CHANGELOG.md` entry (summary / files / reasoning / before-after).
 2. **Never auto-commit changes that touch the valuation/money path** (`target_engine.py`, `finance_data.py`, `generate_model.py`, `run_thesis.py`, `run_socratic.py`, `kill_gate.py`, `model_d*.py`). Propose → human approves the commit. Tests + an LLM review are not sufficient to ship valuation logic unattended.
 3. **Cost-multiplying changes ship env-gated and default-off**, so behavior/cost is unchanged unless explicitly enabled.
 4. **Model tiering is settled:** judgment layer (Socratic A/B/C + corpus callosum + thesis) = `claude-opus-4-8`; deterministic extraction/eval (scouts, `kill_condition_eval`, `event_reasoner`, `generate_model` primary) = `claude-opus-4-6` (honors `temperature=0`); routing = Haiku; research grounding = Perplexity; YouTube = Gemini. **opus-4-8 rejects the temperature param** — always send LLM calls through `utils.create_message`, which strips it on the deprecation error.
 5. **Pure logic is unit-tested; LLM/DB/network calls are guarded** (a failure must never abort a run). Follow the existing module pattern.
 6. **pytest scripts/test_*.py must stay green.** One known failure is environmental: `test_engine_fixtures::test_skip_floor` needs live provider data (skips → floor trips) — it passes with network; do not "fix" it by gutting the floor.
-

1. Current state (this session)

Shipped & tested, NOT yet committed, migrations NOT yet applied. Test suite: **132 passing** (+ the 1 network-gated failure above).

New modules (each with a `test_*.py`): `checkpoint_seal.py`, `run_checkpoint.py` (grader), `kill_gate.py` (D1), `model_d.py`, `model_d_generate.py`, `vix.py`, `analyst_panel.py`, `stability_check.py`. Native scheduler under `scripts/cron/`. Migrations: `supabase/2026-06-01_checkpoint_seal.sql`, `2026-06-03_theses_kill_gate_override.sql`, `2026-06-23_theses_model_d_bracket.sql`.

Edited: `run_socratic.py` (seal hook, opus-4-8 + `create_message` fallback, `[VIX]` / `[CHAIN]` injection, `build_context(include_chain=)`), `run_thesis.py` (kill gate, Model D, `[ARCHETYPE_OVERRIDE]` injection, live spot, drift-strip loop), `finance_data.py` (D2 EDGAR cross-check, `fetch_live_price`, trajectory→advisory), `target_engine.py` (V2 archetype-routed `dcf_role`), `prediction_logger.py`

(seal fields, `actual_date_used`), `utils.py` (`create_message`, `load_env` reused). Fixed a pre-existing syntax error in `discovery_ingest.py`.

Key measured finding (drives P1): `stability_check.py LITE --k 5` shows Model A 100% stable but **Model B (regime) only 60% consistent** chain-off (all three verdicts in 5 runs). Because opus-4-8 has no temperature control, sealed verdicts are partly sampling noise — which corrupts the grader/calibration loop. The `[CHAIN]` context *raised* B to 80% and shifted it (chain earns its place).

2. PRD — work items

P1 — Self-consistency (mode-of-N) for the Socratic round ← highest priority

Problem. Model B's regime verdict is 60% reproducible (measured). A single sealed verdict is partly a dice roll; the grader/`directional_lean`/calibration all inherit that noise. opus-4-8 can't be made deterministic via temperature.

Requirement. Dampen round-1 verdict variance by sampling each model N times and sealing the consensus. Env-gated, default-off (N=1 → today's behavior, zero cost change).

Design.

- New env `SELF_CONSISTENCY_N` (int, default 1). When `>1`, `run_round_1` runs each of A/B/C N times.
- New PURE aggregator (new module `consensus.py`, or extend `stability_check`): given N parsed model outputs, return `{verdict: <modal>, confidence: <modal or worst>, target_low/high: <median across samples sharing the modal verdict>, consistency: <modal frequency 0..1>, n_samples}`. Reuse the `modal_consistency` logic. Categorical fields → mode; numeric fields → median; record `consistency`.
- Wire into `run_socratic.run_round_1_parallel / run_one_model`: if `N>1`, sample N (can parallelize), aggregate, and put `consistency` into the parsed dict so it flows to the seal's `reasoning_fingerprint`.
- Corpus callosum still runs **once** on the aggregated A/B/C.
- Cost: `N ×` the round-1 calls only (CC/research/target unchanged). Document in the module docstring.

Acceptance criteria.

- `SELF_CONSISTENCY_N=1` → byte-identical behavior; full suite stays green.
- `SELF_CONSISTENCY_N=5` → re-run `stability_check.py LITE --k 5`: the **sealed** (aggregated) B verdict is stable run-to-run (modal), and `reasoning_fingerprint` carries a `consistency` field per model.
- Unit tests for the aggregator: mode of categorical verdicts, median of numeric targets, consistency = modal frequency, tie handling, all-different case.
- The seal records per-model `consistency` (so the grader can later down-weight low-consistency seals).

P2 — Validate this session's build live + persist it

Requirement. Everything built this session is unverified against the live stack and uncommitted. Bring it to a known-good, committed state.

Design / steps.

- Apply the 3 migrations via the Supabase SQL editor (all `ADD COLUMN IF NOT EXISTS`, safe):
`checkpoint_seal` fields on `prediction_log` + `prediction_outcomes.actual_date_used`;
`theses.kill_gate_override`; `theses.model_d_bracket`.
- Run the live smoke set (see QA §4) and confirm expected log lines.
- Commit in logical milestones (seal+grader, model upgrade+fixes, D2, V2+D1, VIX, Model D, analyst panel, stability) — **human-approved** per ground rule 2.

Acceptance. Migrations applied; live smoke passes; working tree committed; `git rev-parse --short HEAD` updates so `system_version` on new seals is honest.

P3 — Model D live validation (it's wired; confirm + optionally surface)

Requirement. `run_thesis` already computes a Model D bracket for transformational names (additive, gated, guarded). Confirm it behaves and decide whether to surface it in the dashboard.

Acceptance. A live `run_thesis LITE` prints `[model_d] vision ceiling $X vs engine floor $Y (Nx)`, writes `theses.model_d_bracket`, and leaves conviction/target unchanged. Scenario inputs come from the model's research (not hand-set). (Dashboard surfacing is optional/out-of-scope for now.)

P4 — Socratic-path kill routing (D1 follow-on, #3)

Requirement. `kill_gate.apply_kill_gate` currently runs only in `run_thesis`. Apply the same archetype-routed kill threshold to the Socratic path (Model B / the judgment card), reusing `kill_gate.py`. Carry the structured `kill_gate_override` into the `prediction_log` seal's `reasoning_fingerprint` (so the grader reads the routed verdict). **Touches the live judgment layer — design + review before shipping; do not bundle with P1.**

Acceptance. Defined per the eventual mini-spec; unit-tested gate logic; live confirmation on a transformational name.

P5 — `run_macro.py` (macro cron; VIX as its first input)

Requirement. The macro layer is a single hand-seeded `macro_environment` row. Build the daily orchestrator that refreshes it, with `vix.fetch_vix()` as the first automated signal (regime + level persisted). Add to the native scheduler.

Acceptance. `run_macro.py` writes/updates `macro_environment`; VIX regime persisted; wired into `scripts/cron/crontab.txt`; guarded; pure parts tested.

Deferred backlog (context, not specced here): D3 (timing routing), D4 (tactical/options signals), E1/E3 investigations, F-series (Model D dashboard, lateral-trace discovery, Spark mode, position-sizing-by-maturity), frontend v2 migration, the verdict-bias audit across the watchlist, expand the analyst panel beyond 7 nodes.

3. Todo list (prioritized, actionable)

Now



P1 build `consensus.py` mode-of-N aggregator + unit tests (PURE first).



P1 wire `SELF_CONSISTENCY_N` into `run_socratic.run_round_1_parallel`; thread consistency into the parsed output + seal fingerprint.



P1 validate: `SELF_CONSISTENCY_N=5 python stability_check.py LITE --k 5` shows stable aggregated B.

☐

P2 apply the 3 SQL migrations (Supabase editor).

☐

P2 run the live smoke set (QA §4) and eyeball expected lines.

☐

P2 commit in milestones with CHANGELOG entries (human-approved).

Next

☐

P3 live-validate Model D on LITE/AMD.

☐

P5 build `run_macro.py` + schedule it.

☐

Install the native scheduler (`scripts/cron/crontab.txt`), delete the Claude-app grader task.

Then (needs a mini-spec each)

☐

P4 Socratic-path kill routing.

☐

Verdict-bias audit across the watchlist (measure the tilt).

☐

Expand analyst panel node set; consider dashboard surfacing of `[CHAIN]` + Model D.

4. QA / acceptance checklist

Regression (run after every change):

```
cd scripts && python -m pytest test_*.py -q
# expect: pass, except test_engine_fixtures::test_skip_floor (network-gated; ok offline)
```

Per-feature acceptance:

Feature	Command	Expect
Migrations applied	(Supabase SQL editor)	<code>prediction_log</code> has <code>system_version/reasoning_fingerprint/cohort_key/version_cohort_break/socratic_analysis_id</code> ; <code>prediction_outcomes.actual_date_used</code> ; <code>theses.kill_gate_override</code> , <code>theses.model_d_bracket</code>
Analyst panel	<code>python scripts/analyst_panel.py</code>	7 nodes populated (not "unknown"); <code>data/analyst_panel.json</code> written
Socratic full path	<code>python scripts/run_socratic.py LITE</code>	<code>[VIX] ...</code> , <code>[chain]</code> loaded ..., seal line <code>[checkpoint_seal]</code> sealed LITE (<code>ver=...</code> , <code>cohort=...</code> , <code>break=...</code>)
Thesis full path	<code>python scripts/run_thesis.py LITE</code>	archetype override: <code>transformational ...</code> , <code>[kill_gate] ...</code> , <code>[model_d]</code> vision ceiling ... vs engine floor ..., live spot= (not stale)
Grader	<code>python scripts/run_checkpoint.py --dry-run</code>	grades ripe seals or "no ripe predictions" (none until ~July)
Stability / P1	<code>python scripts/stability_check.py LITE --k 5</code> then <code>SELF_CONSISTENCY_N=5 ...</code>	baseline B≈60%; with N=5 the aggregated verdict is stable
Cron		appends to <code>data/cron/grader.log</code> with OK marker

Feature	Command	Expect
	scripts/cron/ stockradar_cron.sh grader	

P1 unit tests (must add): aggregator mode/median/consistency/tie/all-different;
SELF_CONSISTENCY_N=1 is a no-op.

5. Open decisions for Hume (resolve before P1 ships)

- **N** for self-consistency: recommend **5**.
 - **Scope:** all three A/B/C (recommended; A confirms cheaply) vs B-only (cheaper).
 - **Enable by default or stay env-gated?** Recommend env-gated; enable for sealed/production runs only.
-

Companion docs already in `docs/`: `SYSTEM_STATUS_2026-06-01`, `CHECKPOINT_FEEDBACK_ASSESSMENT/RESOLUTIONS`, `D1_DECISION_BRIEF`, `SESSION_BUILD_REPORT`. Full per-change history in `CHANGELOG.md`.